Backend Case Study (Java)

Partner Event Gateway

Context

Your company is a **B2B platform company working with multiple commerce and logistics partners**. These partners need a standard way to send operational events to your platform.

You are expected to design and implement a backend solution for **Partner Event Gateway**. The goal of this platform is to securely receive, store, process, and expose operational events sent by external partners. The platform must support multiple partners (**tenants**) and ensure proper **security**, **tenant isolation**, **reliability**, and **scalability** designed through production-ready mindset.

No UI is required.

Supported Event Types

The platform must support the following event types:

- OrderCreated
- ShipmentStatusUpdated
- ReturnRequested
- DeliveryAddressUpdated
- OrderCancelled

The platform is **not** expected to implement full downstream business processes for these events. The focus is on **event ingestion and processing platform design**.

Access Scope

The platform serves two different access scopes:

- **External partners** submit events to the platform and may query only **their own events and their latest statuses**.
- **Internal users** may query **all partners' event data** across the platform.

You do **not** need to implement a full internal user authentication or role model for this case. Please keep the focus on **partner-facing security**, **tenant isolation**, and the **core event platform design**. If necessary, you may define reasonable assumptions for different querying personas.

Functional Requirements

1. Partner Support

The system must support multiple external partners.
Each partner must be uniquely identifiable and managed separately.
A partner must not be able to submit events on behalf of another partner.

2. Event Submission

Partners should be able to submit events through an externally exposed **HTTP-based integration interface**.

For each incoming event, the system should:

- identify the partner
- validate the request
- validate the event type
- persist the event reliably
- assign and track a processing status

The exact payload structure is intentionally left open. Please define and document a reasonable model.

3. Event Lifecycle

Each accepted event should have a clear lifecycle.
At minimum, it should be possible to determine whether an event:

- was received
- is pending processing
- was processed successfully
- failed during processing

4. Duplicate Handling

The system must safely handle repeated submission of the same event and prevent incorrect duplicate processing.

5. Querying

The system should support querying with:

- pagination
- configurable page size
- dynamic filtering

Supported filters should include one or more of the following:

- partner
- event type
- event status
- creation/event date interval
- business reference identifier of your choice
- processing outcome

The design should be extensible so that new filtering criteria can be added later.

6. Auditability

It should be possible to trace what happened to an event, including:

- when it was received
- which partner sent it
- its type
- its current processing state
- whether processing succeeded or failed

Non-Functional Expectations

Your solution should address the following:

- **Security:** only authorized partners can submit events; requests should be protected appropriately; secrets/credentials should be handled securely.
- **Tenant Isolation:** one partner must not access or affect another partner's data.
- **Reliability:** accepted events should not be lost because of application failure, retries, timeouts, or temporary internal failures.
- **Idempotency:** duplicate submissions should be handled safely.
- **Concurrency:** the design should account for concurrent submissions, concurrent processing, race conditions, and consistent state transitions.
- **Availability & Performance:** explain how the solution could support high availability, growing traffic, and efficient reads/writes.
- **Maintainability:** the system should be reasonably extensible for new event types, filters, and future evolution.

Design Guidance

To keep the case comparable across candidates, some parts are predefined and some are left open.

Predefined

- the business context

- multi-tenant partner support
- the five event types above
- external submission over HTTP
- partner-level querying of own events and latest statuses
- internal querying across all partner data
- expectations around security, tenant isolation, idempotency, concurrency, reliability, and performance

Open for Your Design Decisions

The following are intentionally left to your judgment:

- API design
- request/response model
- payload structure
- data model
- service boundaries
- modular monolith vs microservice approach
- storage choice
- async processing strategy
- retry/error handling approach
- observability
- deployment/scaling approach

Please make reasonable assumptions and document them clearly.
Please avoid implementing capabilities that are outside the stated scope unless they are necessary to justify your design decisions.

Deliverables

1. Source Code

A working backend implementation in **Java**.
Spring Boot is preferred but not mandatory. You may use any Java-based framework of your choice.

2. README

Include:

- how to build and run the project
- assumptions
- limitations / missing parts
- time spent
- main design decisions and trade-offs

3. API Documentation

Provide a clear API contract, for example using OpenAPI / Swagger.

4. Architecture Documentation

Include:

- a high-level architecture diagram
- main system components
- storage / processing approach
- how your design addresses security, tenant isolation, idempotency, concurrency, reliability, availability, and performance

5. Supporting Diagrams / Model

Include:

- a basic data model / ERD or equivalent
- at least one sequence or flow diagram for event submission and processing

6. Tests

Include unit and integration tests where applicable.

7. API Collection

Include a working Postman collection.

Notes

- You may use any database or supporting technologies you find appropriate.
- You may use synchronous and/or asynchronous processing patterns if appropriate.
- Internal user authentication and role model implementation are not required for this case.
- Clearly document assumptions and intentional simplifications.

Submission

Please send your solution as a zip file including source code, documentation, API collection, and any additional material required to review and run the project.